GeoMatch™ Connecting people to places | ipl immigration policy lab

Building Shiny Apps

Prepared by Amor Ai
June 2026

Introduction

This document serves as a guide for anyone interested in building Shiny applications. It explains what Shiny is, why you might want to build an app, and how to go from a blank script to a tool for teams that need to explore data, validate models, compare policy scenarios, and make decisions together. The examples are drawn from internal GeoMatch apps but the principles apply to any project.

Why Build a Shiny App?

Shiny is an R package (or Python framework) that makes it easy to build interactive web applications (apps) without front-end web development languages like HTML, CSS, or JavaScript. A Shiny app lets users change inputs, filter data, compare options, and see results immediately. More specifically, an app lets you:

- **Explore instead of re-run.** Replace “can you re-run that with a different filter?” round-trips with a dropdown the user controls themselves.
- **Put analysis in non-coders’ hands.** Program staff, partners, and decision-makers can interrogate the data without touching R or waiting on the analyst.

- **Standardize a workflow.** If five people produce the same report by hand, an app turns it into one button, removing copy-paste errors and version drift.
- **Make data legible.** Distributions, comparisons, and trade-offs are far easier to grasp from an interactive chart than from a spreadsheet.
- **Support decisions in real time.** During a meeting, you can answer "what if we changed this assumption?" on the spot.

> **When NOT to build an app**
> Do not build an app just because it is possible. When an analysis runs once and is read once, a rendered document (HTML or PDF) is faster to build and easier to pass on. Reach for Shiny when interactivity, changing inputs, or repetition justifies the added effort.

How Shiny Works

A Shiny app has two main parts: a user interface and a server. The UI defines what the user sees. The server defines what the app does when inputs change.

- **UI** (`ui`) — what the user sees: the layout, inputs (dropdowns, sliders, buttons), and the slots where outputs will appear.
- **Server** (`server`) — the logic: it reads inputs, does the calculations, and builds the outputs.

A call (`shinyApp`) then stitches the two together and launches the app.

Reactivity

Shiny code does not run top to bottom. It runs **reactively**: if a user changes an input, any output that depends on that input updates automatically. This is powerful, but it also means that the order of code in the file is not always the order in which code runs. Think in dependencies: which inputs feed which calculations, and which calculations feed which outputs?

```
input$region        # what the user selected
filtered_data()     # a reactive calculation based on that selection
output$plot         # a rendered result based on filtered_data()
```

Three building blocks cover most needs:

- `reactive()` — a value that recomputes when its inputs change and caches the result. Use it for any intermediate data that more than one output depends on.
- `observeEvent()` — runs side-effecting code in response to a specific event, typically a button press.
- `render*()` (e.g. `renderPlot()`, `renderTable()`) — builds an output and re-builds it when its inputs change.

And two guards you will use constantly — `req()` and `validate()` — are covered below.

Getting Started

What you need

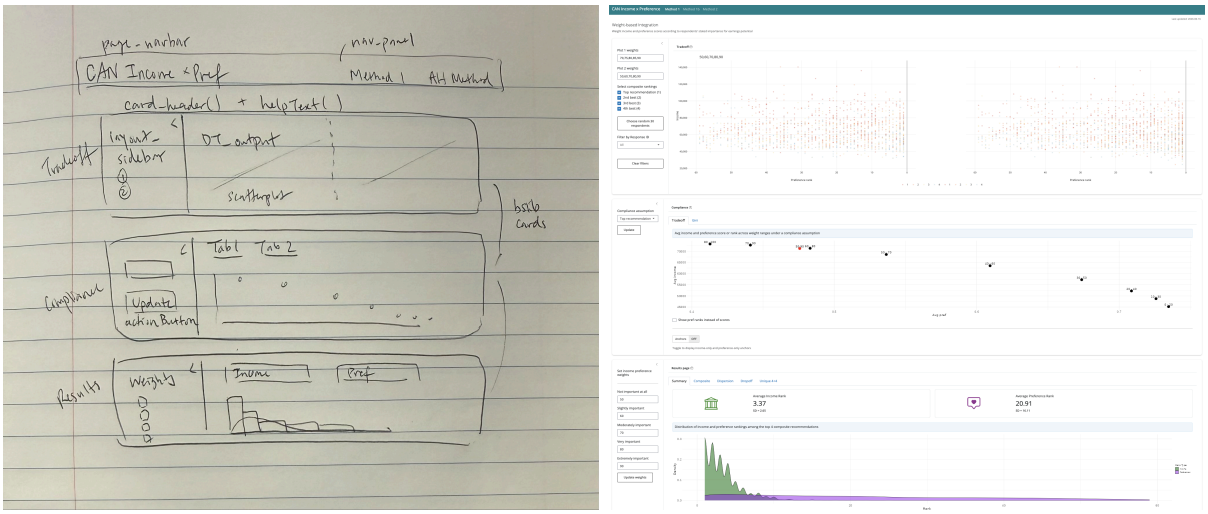
- **R and RStudio/IDE:** RStudio has Shiny support built in — new file → Shiny Web App gives you a working template. Other IDEs like Positron also work well.
- **The shiny package:** `install.packages("shiny")`.
- **A dataset (or several).**

Sketch the design before you write code

Before writing any code, sketch the UI you have in mind on paper. Block out where the inputs go and how the outputs are presented, then work backwards from that final picture to the inputs it requires. Sometimes you'll want a left sidebar because the output — a graph, a table, and so on — is more vertical. Other times you'll want a top bar for inputs because your output is more horizontal. If you are new to Shiny, it helps to browse the widget gallery first to see what layouts and controls are available.

Note on prompting LLMs

LLMs have made building an app dramatically faster, but it is still worth having a clear idea of what you want before asking one for help, rather than building from scratch and iterating from a vague starting point. It is far more efficient to say "three `bslib` cards filling the page, with a select input on the left above the second card" than "build an app with this data."



First sketch vs final layout

Writing code others can read

Make your code readable. Use proper indentation and newlines. Name objects after what they are — `can_metrics_df`, not `data1` or `df2`. Comment liberally so others can build on your code, and split the app into clearly separated sections.

Functions are your best friend. They cut redundancy, and redundancy is what makes an app confusing to read. If only one element of a table or plot changes — say, which region's data it shows — do not copy the block and tweak it. Write one function that takes the changing piece as an argument, and call it with different values.

For a concrete example, imagine a tool that builds a summary card for each of eight economic regions. The cards look identical; only the underlying data differs. Rather than writing the card code eight times, write it once as a function and feed it each region:

```
observeEvent(input$create_cards, {
  req(region_data())

  data <- region_data() |>
    filter(admission_category == input$category) |>
    filter(region_name %in% input$regions_selected)

  # One function, called per region — not eight near-identical blocks
  cards$card1 <- create_region_card(data, region_rank = 1)
  cards$card2 <- create_region_card(data, region_rank = 2)
  cards$card3 <- create_region_card(data, region_rank = 3)
  # ... and so on
})
```

In an ideal world, functions are nearly the only things inside your server: the server calls functions that do the real work, and those functions live somewhere more accessible (for example, an R/ folder or a `functions.R` file) rather than buried in server code. That is not always feasible — even the example above could be vectorized to create all eight cards from one line — but generally, functions save you and others a great deal of headache later.

Put the app under version control

Create a Git repository before writing substantial code. Version control gives you a history of changes, makes it easy to collaborate with others, and provides a safety net when experiments go wrong.

Example of a minimal complete app

```
library(shiny)
library(tidyverse)   # data manipulation + ggplot2
library(plotly)      # interactive plots


# Load the data once, at startup. Reading it outside the server means
# it loads a single time when the app launches, not once per user.
can_metrics_df <- read_csv("data/metrics.csv")


# ---- UI: what the user sees ----
ui <- fluidPage(
  titlePanel("Economic Region Metric Explorer"),
```

```
  sidebarLayout(
    sidebarPanel(
      # One control: choose which metric to display
      selectInput(
        inputId = "metric",   # referenced server-side as input$metric
        label   = "Choose a metric",
        choices = sort(unique(can_metrics_df$metric))  # available metrics
      )
    ),
    mainPanel(
      plotlyOutput("metric_plot"),  # slot for the bar chart
      tableOutput("top_regions")    # slot for the top-10 table
    )
  )
)


# ---- Server: turns inputs into outputs ----
server <- function(input, output, session) {


  # Filter to the chosen metric once, then reuse. Both outputs depend on
  # this, so doing it here avoids repeating the filter in each render block.
  selected_metric <- reactive({
    req(input$metric)              # wait until a metric is selected
    can_metrics_df |>
      filter(metric == input$metric)
  })


  # Interactive bar chart: score by region, ordered high to low.
  output$metric_plot <- renderPlotly({
    p <- selected_metric() |>
      ggplot(aes(x = score, y = reorder(region_name, score))) +
      geom_col(fill = "#07838C") +
      labs(x = "Score", y = NULL)


    ggplotly(p)   # hand the ggplot to plotly for zoom / hover / pan
  })


  # Companion table: the ten highest-scoring regions for that metric.
  output$top_regions <- renderTable({
    selected_metric() |>
      arrange(desc(score)) |>
      select(region_name, score) |>
      head(10)
  })
}


shinyApp(ui, server)   # launch the app
```

UI/UX Considerations

The following practices are mostly drawn from the User Feedback chapter of *Mastering Shiny* (Wickham, 2021).

Load the page as fast as possible

People hate waiting, and they especially hate a blank screen that gives no sign anything is happening.

Best practice: render as much of the UI as possible immediately, while data, model objects, and queries load in the background. Even just showing the sidebar and a loading animation cuts frustration dramatically.

One way to do this is to wrap data loading in a reactive block that waits on something in the UI, using `req()`. The block does not run until the thing it requires exists — which forces the UI to appear first:

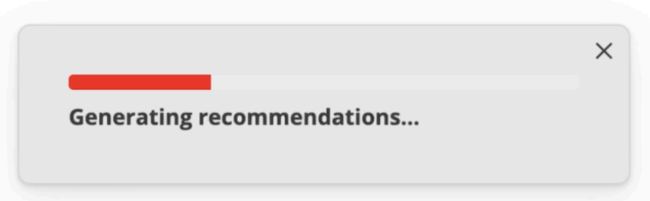
```
candidate_data <- reactive({
  req(input$cohort)       # waits until the cohort input exists

  if (input$category == "Economic") {
    get_economic_cases(cohort = input$cohort, year = input$year)
  } else {
    get_other_cases(cohort = input$cohort, year = input$year)
  }
})
```

If `input$cohort` has a default value, this block fires as soon as that value exists (and again whenever it changes), so the UI must load first. If it has no default, the block waits until the user makes a selection.

Show progress: loading bars and spinners

Just as with page load, users hate not knowing whether something is working. Add a progress bar when the runtime or number of iterations is known, or a loading spinner when it is not. The shinycssloaders package adds a spinner to any output in essentially one line.



Example of a progress bar

Tell users what changed: update messages

When you ship changes, let users know with an update notification. Summarize the important changes and set a sensible duration. HTML formatting lets you add line breaks and bold text:

```
observe({
  showNotification(
    HTML(paste(c("<b>APP UPDATE (NEW DATE)</b>",
                 "<b>Bug fixes:</b>",
                 "Corrected income-score rounding in the region table"),
               collapse = "<br/>")),
      duration = 10,
      type = "message"
  )
})
```

Handle errors gracefully

Some errors break the app outright; others surface an ugly, incomprehensible red message to the user. Avoid both with `validate()` and standard R error handling like `tryCatch()`.

Suppose a user selects a region or cohort that has no data. Instead of letting every downstream calculation fail, stop early with a `validate()` statement and show a human-readable message. `req()` only checks that an object exists, while `validate()` checks any condition you specify.

```
output$income_plot <- renderPlot({
  req(region_data())
  validate(
    need(nrow(region_data()) > 1,
       message = "No data available for this selection.")
  )

  # ... plotting code runs only if the data is sufficient
})
```

Now, rather than a red error, the user sees a clear message pointing at where the problem lies.

Minimize redundancy for the user

The function-oriented principle applies to the interface too. If two views are similar enough, make them dynamic instead of duplicating them. For example, instead of a separate tab for every admission category, use one tab with a dropdown that lists the available categories and updates the table or plot accordingly.

There are limits: if one category needs an extra chart or a genuinely different layout, a dedicated tab is fine. In general, start as dynamic as possible and add tabs only when a view truly diverges — rather than starting with repetitive tabs and trying to consolidate later.

Be accessible — especially about color

Color blindness isn't uncommon, and depending on your audience, it may be more common than in the general population. Avoid encoding good/bad as red/green — that is the most common form of color blindness. The `viridis` package provides color-blind-safe palettes that also reproduce well in grayscale. For more, see the Appsilon accessibility guide.

Recommended Packages

You can build a great deal with base `shiny` and `ggplot2`, but there are quite a few other handy packages.

UI, layout, and inputs

- **`bslib`** — docs. A personal favorite way to lay out and theme an app: cards, sidebars, value boxes, and Bootstrap 5 theming with light/dark and custom palettes.
- **`shinythemes`** — docs. An easy way to alter the appearance of your app via `shinytheme()`.
- **`shinyWidgets`** — docs. A large set of better input controls than base Shiny. Run `shinyWidgets::shinyWidgetsGallery()` in the console to browse them interactively — `pickerInput()` and grouped buttons are standouts.
- **`shinycssloaders`** — docs. One-line loading spinners (see the UI/UX section).
- **`shinyjs`** — docs. Less commonly needed, but the ability to enable/disable inputs is very handy — e.g. disabling a button when the data it depends on is unavailable.
- **`shinydashboard`** — docs. Gives you tools to create visual "dashboards".

Plotting

- **`ggplot2`** — docs. The foundation for nearly every static plot. Worth being fluent in before reaching for anything fancier.
- **`gghighlight`** — docs. Makes specific plot elements stand out. Useful for contextualizing one selected case among a population of others.
- **`ggdist`** — docs. Excellent for distributions and uncertainty, including beeswarm plots — bars, a density curve, or every individual point (the last is great in interactive settings where a user can hover). Works well for the distribution of income scores among candidates matched to a region.
- **`ggrepel`** — docs. A near must-have for labeling points: labels repel each other to avoid overlap.
- **`geomtextpath`** — docs. Labels lines directly on the plot instead of in a legend, which often just gets in the way.
- **`scattermore`** + **`locfit`** — scattermore rasterizes hundreds of thousands of points quickly; locfit fits smoothed trend lines much faster than the default `method="loess"` in `geom_smooth()`. Use them together for large scatter plots.

Interactive plotting

- **`plotly` / `ggplotly`** — docs. The most straightforward way to make a `ggplot` interactive: build the plot in `ggplot`, then wrap it in `ggplotly()` for zoom, pan, hover, and point selection out of the box. This is the path of least resistance and what the example apps use.
- **`ggiraph`** — docs. More customizable than `ggplotly`, but a bit less convenient for quick use.
- **`leaflet`** — docs. Add interactive maps to your apps.

Tables

- **`DT`** — docs. The most common table package in Shiny and the one the example apps use. It cannot do everything, but it is the most stable option and the easiest to find Stack Overflow answers for.
- **`reactable`** — docs. A great alternative for interactive tables: thorough docs, and a cleaner, more modern default style. See the cookbook for what is possible.
- **`gt`** — docs. Produces beautiful static tables. Its interactivity and Shiny integration are less of a known quantity, but worth a look for polished display tables.

Testing

- **`shinytest2`** — docs. Automated testing for Shiny apps. It runs your app in a headless browser, records snapshots of inputs and outputs, and flags when a change alters behavior.

Tips and Tricks

- **`hr()` adds a horizontal rule in the UI** . Great for separating groups of inputs that are conceptually distinct or meant to be used in sequence.
- **CSS for small fixes.** To adjust one button's color, tighten spacing between two inputs, or add table padding, drop a small CSS block into your UI. Finding the right element name is the tricky part; browser developer tools help.

```
tags$style(
  type = "text/css",
  ".checkbox, .radio { margin-top: 10px; margin-bottom: 1px; }
   .rt-td { cursor: cell; }
   .modal-dialog { width: fit-content !important; }"
)
```

- **Explain an input without cluttering the layout with a clickable info icon and popover beside it.** Position an `icon("info-circle")`over the input and initialize the popover once Shiny connects.
- **Swap data with one toggle instead of duplicating a view.** A single switch can change the underlying dataset — far cleaner than two near-identical tabs:

```
input_switch("show_z", "Show z-scores")
# server:
data_source <- if (input$show_z) z_scores_df else raw_scores_df
```

- **Open the app in a real browser while testing.** This shows the true width and height a user will have, and some features only work outside the RStudio viewer (external URLs that open in a new tab, for instance). Make it the default while developing:

```
shinyApp(ui = ui, server = server, options = list(launch.browser = TRUE))
```

- **Browse better inputs.** Run `ShinyWidgets::shinyWidgetsGallery()` in the console for an interactive menu of input controls — there are many useful upgrades over the base options, such as `pickerInput()` and grouped buttons.

Deploying and Sharing Your App

There are three common ways to put it in front of users; pick based on who needs access and how sensitive the data is.

- **Run locally.** For a one-off or a single user, sharing the code (or a Git repository) so someone runs it in their own RStudio (or chosen IDE) is sometimes enough.
- **shinyapps.io** — docs. The fastest way to share a non-sensitive app via a URL. Be deliberate about what data goes on a hosted service. (Rstudio servers are not approved for high risk data. See Stanford IT Risk Classification for more details.) This site is managed by RStudio and is free for small apps with limited visits and scales up in paid versions.
- **Posit Connect / internal server** — docs. An option for anything touching sensitive or non-public data. The app runs on infrastructure your team controls, behind authentication.

Additional Resources

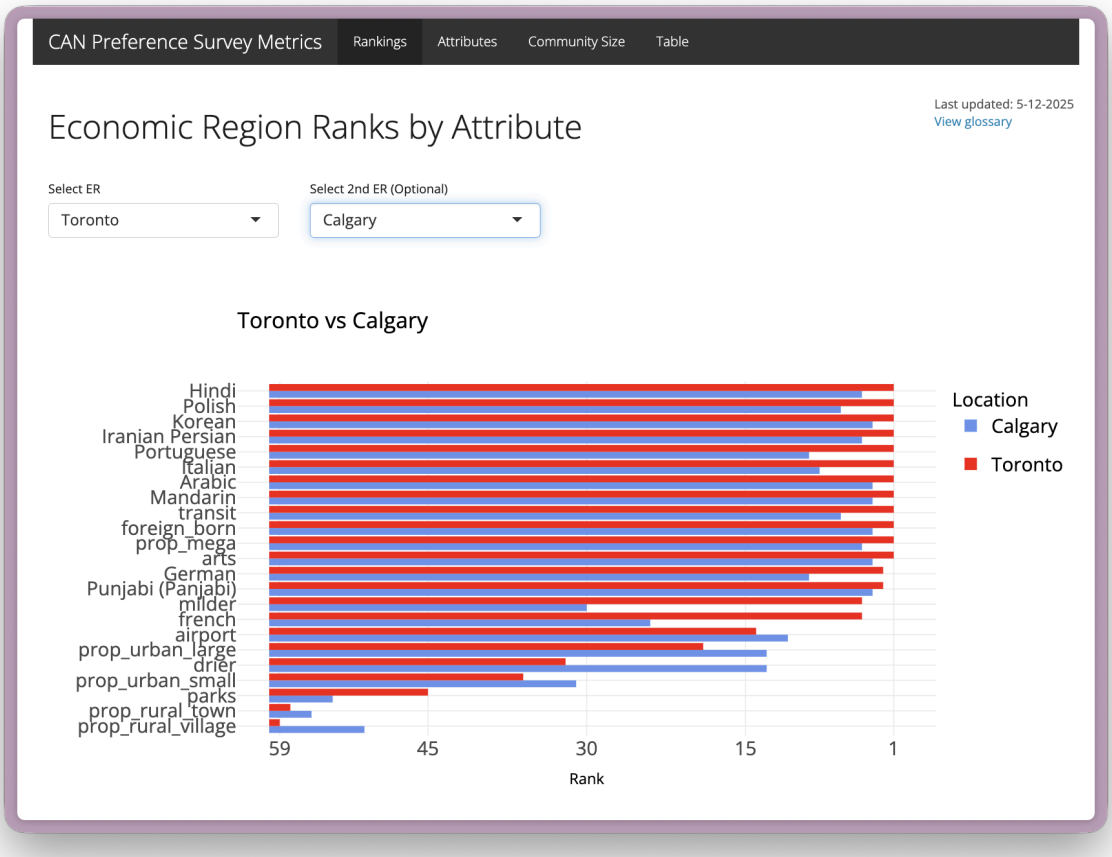
- Mastering Shiny — Hadley Wickham's free book, with all example app code on GitHub. The definitive starting point; the reactivity and user-feedback chapters are essential.
- Shiny official site and gallery — documentation, tutorials, and a gallery of example apps with source code.
- Outstanding User Interfaces with Shiny — David Granjon's book on deeply customizing and enhancing Shiny applications.
- Engineering Production-Grade Shiny Apps — for when an app graduates from a personal tool to shared infrastructure.
- Shiny coding style guide — conventions for readable Shiny code.
- Appsilon accessibility guide — making apps usable for everyone, color blindness included.
- Your favorite LLM (Claude, Codex, etc.)

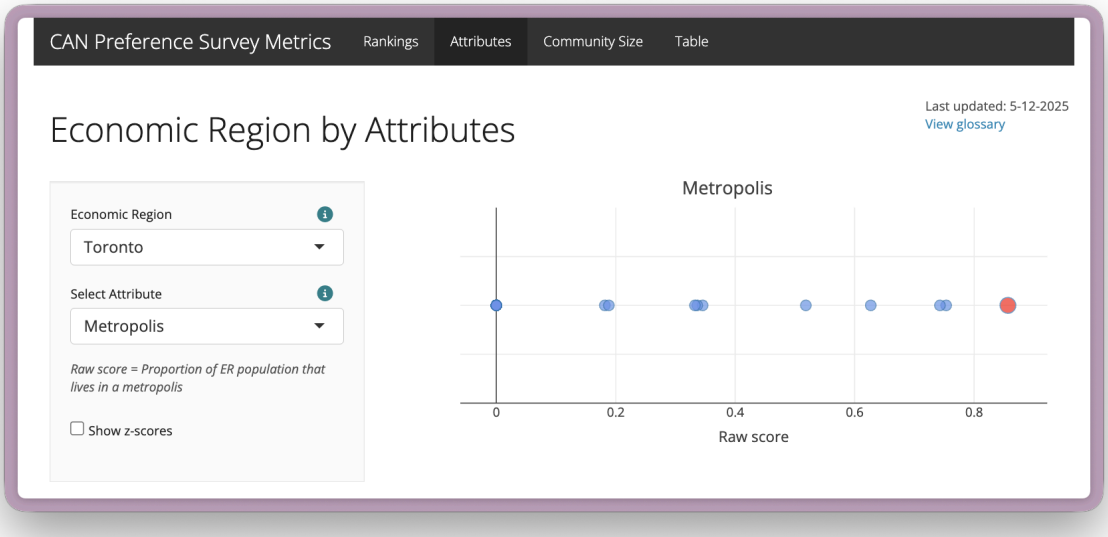
Gallery

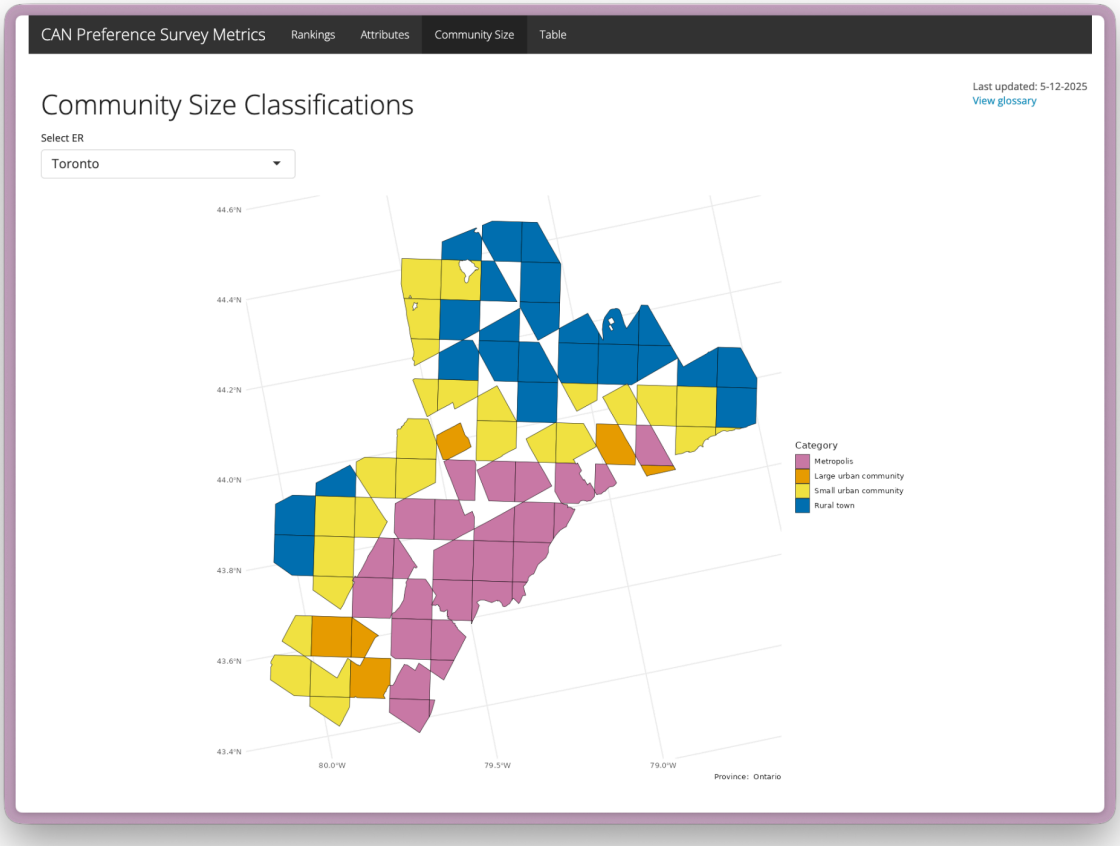
The screenshots below give a sense of what Shiny apps can look like. These tools were built for GeoMatch to support decision-making and the development, validation, and demonstration of the GeoMatch Canada preference and metrics pipelines.

Economic Region Metrics App

A tool used to visualize and validate the metrics scores that feed into the GeoMatch Canada recommendation system. The app allows users to examine raw metric values, scaled scores, rankings, and community size categories across economic regions.

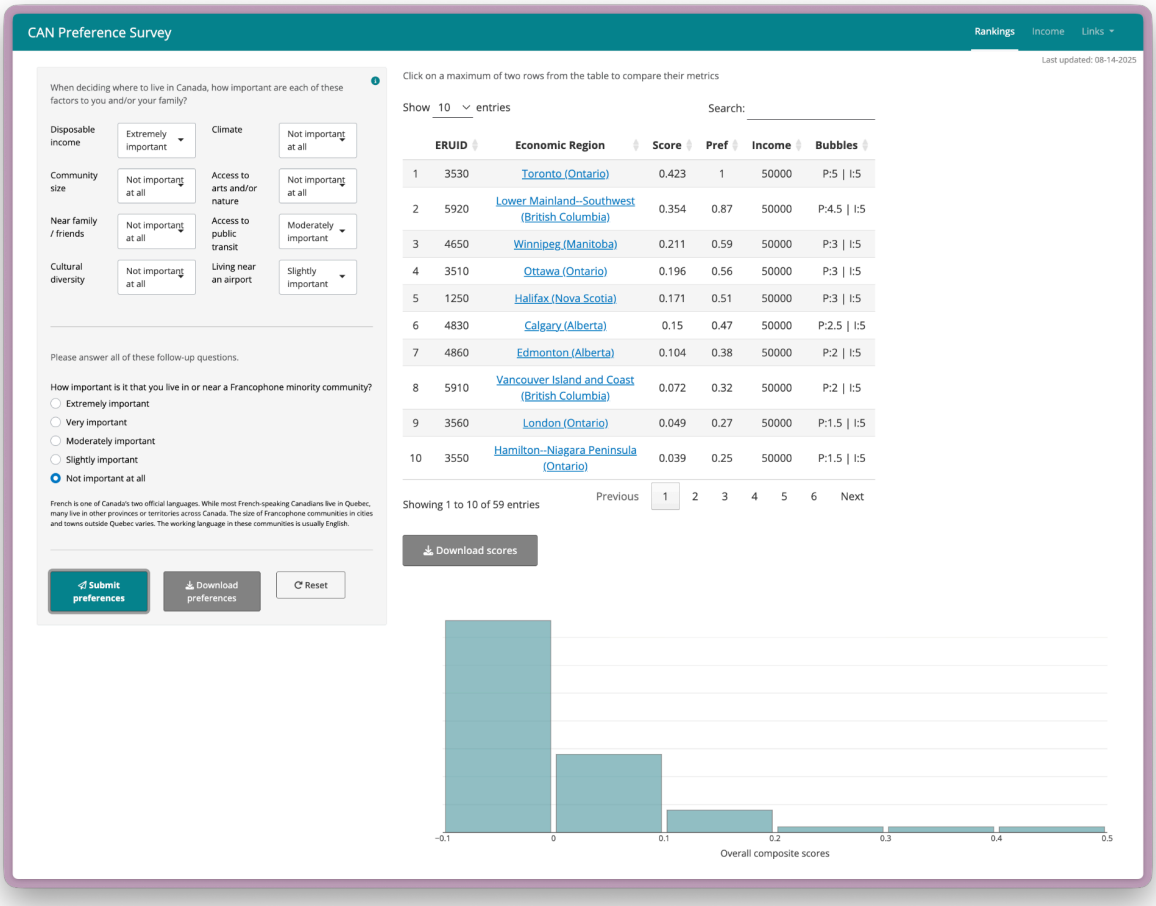



CAN Preference Survey Metrics
Rankings
Attributes
Community Size
Table
Economic Region by Attributes
Last updated: 5-12-2025
View glossary
Economic Region
Toronto
Select Attribute
Metropolis
Raw score = Proportion of ER population that lives in a metropolis
Show z-scores
Metropolis
0
0.2
0.4
0.6
0.8
Raw score


CAN Preference Survey Metrics
Rankings
Attributes
Community Size
Table
Community Size Classifications
Last updated: 5-12-2025
View glossary
Select ER
Toronto
44.6°N
44.4°N
44.2°N
44.0°N
43.8°N
43.6°N
43.4°N
80.0°W
79.5°W
79.0°W
Province: Ontario
Category
Metropolis
Large urban community
Small urban community
Rural town

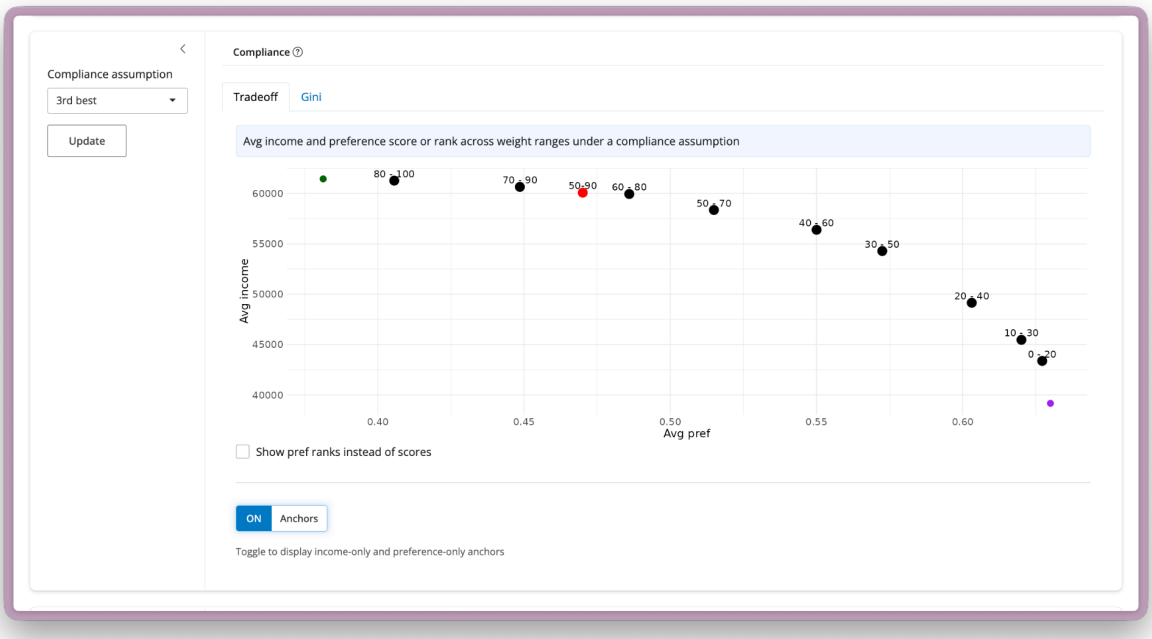
Preference Recommendation App

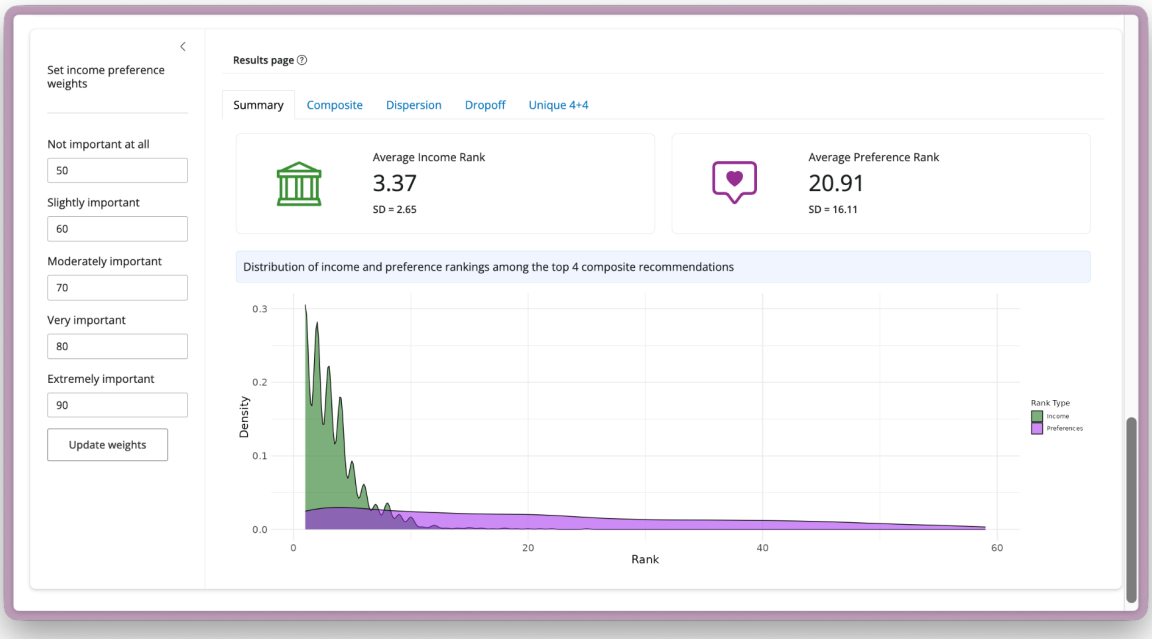
A tool used to visualize how user preference responses translate into ranked location recommendations. It provides an interactive interface for examining preference inputs, reviewing how the aggregation function produces scores, and validating that locally computed preference rankings align with those generated in the production environment.

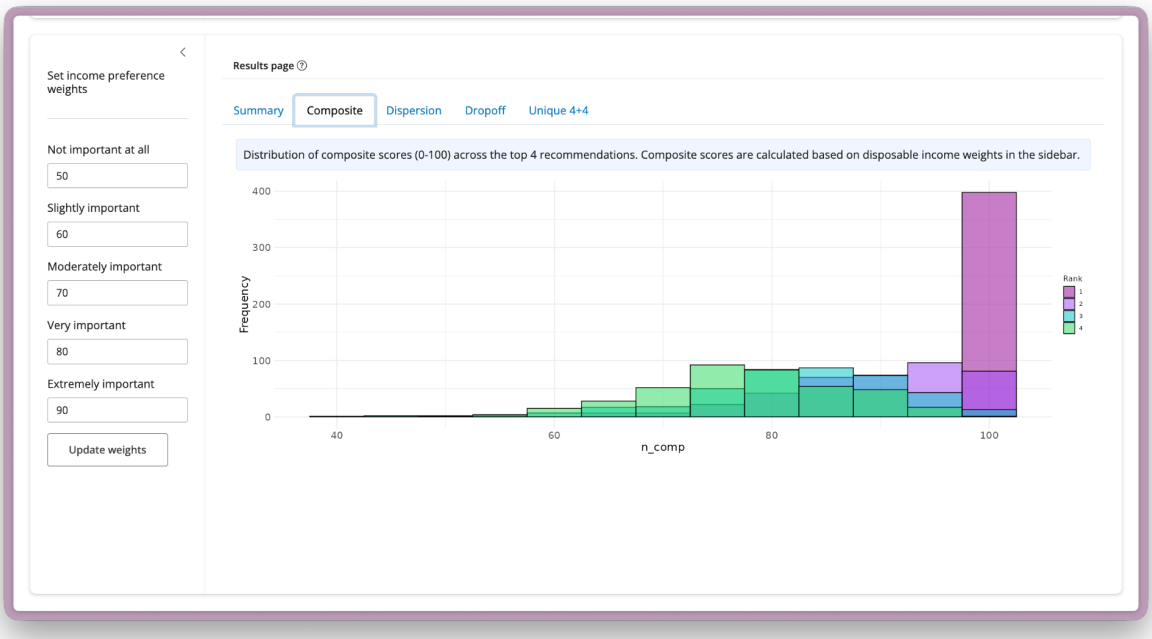


Income x Preference Integration App

A tool used to test and visualize the integration of preference outputs with income predictions. The app allows users to explore different weighting schemes and integration methods, examine resulting rankings, and understand the tradeoffs that emerge when adjusting weights and combining scores.


Compliance assumption
3rd best
Update
Compliance
Tradeoff
Gini
Avg income and preference score or rank across weight ranges under a compliance assumption
80 - 100
70 - 90
50 - 90
60 - 80
50 - 70
40 - 60
30 - 50
20 - 40
10 - 30
0 - 20
60000
55000
50000
45000
40000
Avg income
0.40
0.45
0.50
0.55
0.60
Avg pref
Show pref ranks instead of scores
ON
Anchors
Toggle to display income-only and preference-only anchors


Set income preference weights
Not important at all
50
Slightly important
60
Moderately important
70
Very important
80
Extremely important
90
Update weights
Results page
Summary
Composite
Dispersion
Dropoff
Unique 4+4
Average Income Rank
3.37
SD = 2.65
Average Preference Rank
20.91
SD = 16.11
Distribution of income and preference rankings among the top 4 composite recommendations
0.3
0.2
0.1
0.0
Density
0
20
40
60
Rank
Rank Type
Income
Preferences



This document draws on Justin Perline's Shiny style guide from my time with Pittsburgh Pirates R&D and on Hadley Wickham's Mastering Shiny, with examples reframed around IPL and GeoMatch work. Feel free to reach out to me with any questions.